ASEN 5264 – DMU

Homework #3

Bijan Jourabchi

Problem 1:

a) Given U^* we can extract the optimal policy π^* from U^* with the following expression:

$$\pi^*(s) = \arg\max_{a \in A}(R(s,a) + \gamma \mathbb{E}[U^*(s)])$$

b) We are given an MDP defined by,
$S = \mathbb{R}, \quad A = \{0, 0.1, 1\}, \quad \gamma = 1$
$T(s' \mid s, a) = \mathcal{U}([s+a, s+a+1])$
$R(s,a) = -\mathbb{1}(s \leq 1) - a$
$U^{\tilde{\pi}}(s) = -2 \cdot \mathbb{1}(s \leq 1)$

Where $\mathcal{U}([s+a, s+a+1])$ represents a uniform distribution from $s+a$ to $s+a+1$, and $\mathbb{1}(s \leq 1) = 1$ if $s \leq 1$ and 0 otherwise. Given $s = 0.8$, we must find the action which maximizes $Q^{\tilde{\pi}}(s,a)$, which is given by

$$Q^{\tilde{\pi}}(s,a) = R(s,a) + \sum_{s'} T(s' \mid s, a) U^{\tilde{\pi}}(s')$$

Since there are only three actions, we can compute $Q^{\tilde{\pi}}(s = 0.8, a)$ for $\forall a \in A$. Since, S and T are continous, we also note that the summation will be an integral over all of the possible future state s'. Thus, our expression for $Q^{\tilde{\pi}}(s,a)$ becomes

$$Q^{\tilde{\pi}}(0.8,a) = R(0.8,a) + \int_{-\infty}^{\infty} T(s' \mid 0.8, a) U^{\tilde{\pi}}(s') ds'$$

We can now compute $Q^{\tilde{\pi}}(0.8,a)$ for all a ∈ A, begining with $a = 0$. Note that $\mathcal{U}([s+a, s+a+1]) = 1 \ \forall \ a, s$ by definition of the uniform distribution.

a = 0:

$$Q^{\tilde{\pi}}(0.8,0) = R(0.8,0) + \int_{-\infty}^{\infty} T(s' \mid 0.8, 0) U^{\tilde{\pi}}(s') ds'$$
$$Q^{\tilde{\pi}}(0.8,0) = (-\mathbb{1}(0.8 \leq 1) - 0) + \int_{-\infty}^{\infty} \mathcal{U}([0.8, 1.8])(-2 \cdot \mathbb{1}(s' \leq 1)) ds'$$
$$Q^{\tilde{\pi}}(0.8,0) = -1 + \int_{-\infty}^{\infty} (-2 \cdot \mathbb{1}(s' \leq 1)) ds'$$
$$Q^{\tilde{\pi}}(0.8,0) = -1 + \int_{0.8}^{1} -2 ds' = -1 - 2(0.2) = -1.4$$

a = 0.1:

$$Q^{\tilde{\pi}}(0.8, 0.1) = R(0.8, 0.1) + \int_{-\infty}^{\infty} T(s' \mid 0.8, 0.1) U^{\tilde{\pi}}(s') ds'$$
$$Q^{\tilde{\pi}}(0.8, 0.1) = (-\mathbb{1}(0.8 \leq 1) - 0.1) + \int_{-\infty}^{\infty} (-2 \cdot \mathbb{1}(s' \leq 1)) ds'$$
$$Q^{\tilde{\pi}}(0.8, 0.1) = -1.1 + \int_{0.9}^{1.9} (-2 \cdot \mathbb{1}(s' \leq 1)) ds'$$
$$Q^{\tilde{\pi}}(0.8, 0.1) = - - 1.1 + \int 0.9^1 - 2 ds' = -1.1 - 2(0.1) = -1.3$$

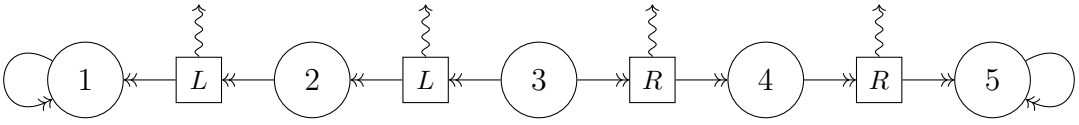
a = 1:

$$Q^{\tilde{\pi}}(0.8, 1) = R(0.8, 1) + \int_{-\infty}^{\infty} T(s' \mid 0.8, 1) U^{\tilde{\pi}}(s') ds'$$
$$Q^{\tilde{\pi}}(0.8, 1) = (-\mathbb{1}(0.8 \leq 1) - 1) + \int_{-\infty}^{\infty} (-2 \cdot \mathbb{1}(s' \leq 1)) ds'$$
$$Q^{\tilde{\pi}}(0.8, 1) = -2 + \int_{1.8}^{2.8} (-2 \cdot \mathbb{1}(s' \leq 1)) ds'$$
$$Q^{\tilde{\pi}}(0.8, 1) = -2$$

Thus the action which minimizes $Q^{\tilde{\pi}}(0.8, a)$ is $\boxed{\text{a} = 0.1}$.

Problem 2:

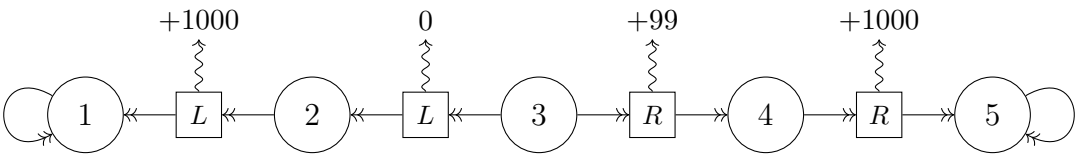
Given the following MDP:



We are asked to disprove the following statement:

> If a policy π satisfies $|Q^*(s, \pi^*(s)) - Q^*(s, \pi(s))| \leq \beta$ for all $s \in S$ for some $\beta > 0$, then it immediately follows that $|R(s, \pi^*(s)) - R(s, \pi(s))| \leq \beta$ for any $s \in S$.

In order to find a counter example, we must find a reward function, R a policy π, and a value β that disproves the claim. Since all transitions are deterministic, and every state other than $s = 3$ only has a single action we only have to choose $\pi(3)$ and $\pi^*(3)$, the optimal policy. When choosing $R(s, a)$, we will ensure that one action at $s = 3$ is clearly better than the other action, that way our policy will differ from the optimal policy which is necessary to disprove the statement. To disprove the claim, consider the following reward function for our MDP:



It is clear that the optimal policy at $s = 3$ is $\pi^*(3) = R$, so we will choose our policy to be $\pi(3) = L$. Next, we will use the Bellman Backup algorithm in order to compute values for $Q(s, \pi)$. Since we have two terminal states, $s = 1$ and $s = 5$, we will start there and work back until we can compute $Q(3, \pi) = R(3, \pi(3)) + 0.9U^{\pi}(s')$ for both the optimal policy and non-optimal policy,

S	A	Q^*	V^*
1	N/A	0	0
5	N/A	0	0
4	R	1000	1000
2	L	1000	1000
3	R	$99 + 0.9(1000) = 999$	999
3	L	$0.9(1000) = 900$	

Thus, we get $Q(3, \pi^*(3)) = 999$ and $Q(3, \pi(3)) = 900$. Finally, let's select $\beta = 99.5$. Our policy satisfies $|Q^*(3, \pi^*(3)) - Q^*(3, \pi(3))| \leq \beta \rightarrow |999 - 900| \leq 99.5 \rightarrow 99 \leq 99.5$. The original claim states that it immediately follows that $|R(3, \pi^*(3)) - R(3, \pi(3))| \leq \beta$. However, we see that $|100 - 0| = 100 \not\leq 99.5$. Thus, we have found a counter example for the claim.

Problem 3:

a) Using the code provided to us, I was able to achieve a mean discounted reward estimate of -95.63 reward units with 260 Monte Carlo trials. To compute the SEM, I used the following formula,

$$SEM = \frac{\sigma}{\sqrt{n}}$$

where σ is the standard deviation of the reward estimate, and n is the number of Monte Carlo trials. With $n = 260$ MC trials, I achieved an $SEM = 4.98$.

b) Given the information that goal cells were 20 cells apart, i.e. at $[20, 20]$, $[20, 40]$, $[40, 20]$, etc. I created a heuristic policy which given a state s, first finds the closest goal cell in the $60x \times 60$ grid, then chooses the appropriate action in order to move towards that goal cell. Code for my heuristic policy can be seen in the smart_policy function below. Using again $n = 260$ MC trials, I was able to achieve a mean discounted reward estimate of -29.30 and a $SEM = 3.75$.

3.1 Code For Problem 3

```
using DMUStudent.HW3: HW3, DenseGridWorld, visualize_tree
using POMDPs: actions, @gen, isterminal, discount, statetype, actiontype,
    simulate, states, initialstate
using D3Trees: inchrome, inbrowser
using StaticArrays: SA
using Statistics: mean, std
using BenchmarkTools: @btime

m = HW3.DenseGridWorld(seed = 3)

function rollout(mdp, policy_function, s0, max_steps=100)
    r_total = 0.0
```

```
    t = 0
    s = s0
    while !isterminal(mdp, s) && t < max_steps
        a = policy_function(mdp, s) # replace this with a policy
        s, r = @gen(:sp,:r)(mdp, s, a)
        r_total += discount(m)^t*r
        t += 1
    end
    return r_total # replace this with the reward
end

### Part a)
function heuristic_policy(m, s)
    # put a smarter heuristic policy here
    return rand(actions(m))
end

function smart_policy(m,s)
    x,y = s

    AS = [
        (:down, (x, y-1)),
        (:up, (x, y+1)),
        (:right, (x+1, y)),
        (:left, (x-1, y))
    ]


    # Find closest goal from CURRENT state
    goals = [[20,20], [20,40], [40,20], [40,40]]
    closest_goal_cell = goals[argmin([sqrt((x - g[1])^2 + (y - g[2])^2)
       for g in goals])]
    #@show s
    #@show closest_goal_cell

    # Find which next state is closest to that goal
    distances_to_goal = [sqrt((next_s[1] - closest_goal_cell[1])^2 + (
       next_s[2] - closest_goal_cell[2])^2) for (a,next_s) in AS]

    #@show distances_to_goal
    #@show AS[argmin(distances_to_goal)][1]
    return AS[argmin(distances_to_goal)][1]

end

# This code runs monte carlo simulations: you can calculate the mean and
   standard error from the results
# Part a)
n = 260

results = [rollout(m, heuristic_policy, rand(initialstate(m))) for _ in 1:
   n]

@show mean(results)
```

```
@show std(results)/sqrt(n)

# part b)
n = 260
results = [rollout(m, smart_policy, rand(initialstate(m))) for _ in 1:n]

@show mean(results)
@show std(results)/sqrt(n)
```

Problem 4:

After 7 iterations of Monte Carlo Tree Search with a depth of 100 and c = 2, we get the tree seen in Figure 1. The actions which lead to the highest Q values from the starting state are the up and right actions. This is because these actions move the state closest the goal cell at $[20, 20]$. Since I used my heuristic policy outlined in problem 3 for the rollout simulation in MCTS, these actions will be selected the most, which we also see in the tree. Note that the value I chose for c was arbitrarily chosen with no tuning. It was tuned in later problems.

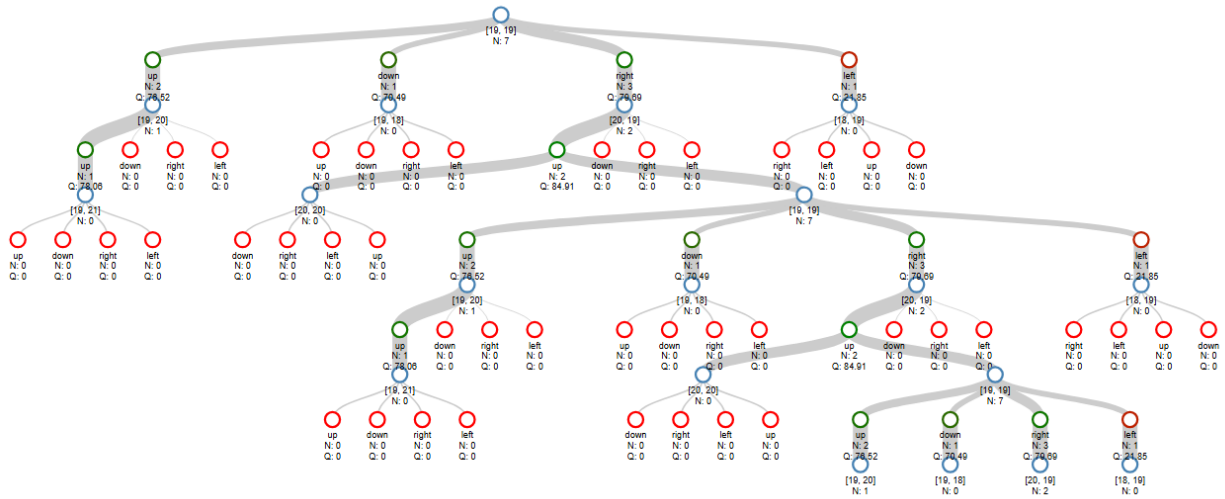


Figure 1: Monte Carlo Tree Seach Decision Tree After 7 iterations

4.1 Code For Problem 4

```

using DMUStudent.HW3: HW3, DenseGridWorld, visualize_tree
using POMDPs: actions, @gen, isterminal, discount, statetype, actiontype,
    simulate, states, initialstate
using D3Trees: inchrome, inbrowser
using StaticArrays: SA
using Statistics: mean
using BenchmarkTools: @btime

############
```

```
# Question 4
############

function rollout(mdp, policy_function, s0, max_steps=100)
    r_total = 0.0
    t = 0
    s = s0
    while !isterminal(mdp, s) && t < max_steps
        a = policy_function(mdp, s) # replace this with a policy
        s, r = @gen(:sp,:r)(mdp, s, a)
        r_total += discount(m)^t*r
        t += 1
    end
    return r_total # replace this with the reward
end



function smart_policy(m,s)
    x,y = s

    AS = [
        (:down, (x, y-1)),
        (:up, (x, y+1)),
        (:right, (x+1, y)),
        (:left, (x-1, y))
    ]


    # Find closest goal from CURRENT state
    goals = [[20,20], [20,40], [40,20], [40,40]]
    closest_goal_cell = goals[argmin([sqrt((x - g[1])^2 + (y - g[2])^2)
       for g in goals])]
    #@show s
    #@show closest_goal_cell

    # Find which next state is closest to that goal
    distances_to_goal = [sqrt((next_s[1] - closest_goal_cell[1])^2 + (
       next_s[2] - closest_goal_cell[2])^2) for (a,next_s) in AS]

    #@show distances_to_goal
    #@show AS[argmin(distances_to_goal)][1]
    return AS[argmin(distances_to_goal)][1]

end

function simulate_MCTS(π, s, d = 100)

    # Unpack
    m = π.m

    if d <= 0 # Base case

        return rollout(m, smart_policy, s) # U(s)
```

```
    end

    A = actions(m)
    γ = discount(m)

    # If we haven't checked the actions at state s yet, assign all dict
       values to 0
    if !haskey(n, (s, first(A)))
        for a in A
            π.n[(s,a)] = 0
            π.q[(s,a)] = 0.0

        end
        # Value funciton estimate
        return rollout(m, smart_policy, s)
    end

    # Explore decision tree
    a = explore(π,s)
    sp, r = @gen(:sp,:r)(m, s, a)
    Q = r + γ * simulate_MCTS(π, sp, d-1)

    π.n[(s,a)] += 1
    π.q[(s,a)] += (Q-π.q[(s,a)])/π.n[(s,a)]
    tval = get(π.t, (s, a, sp), 0)
    π.t[(s,a,sp)] = tval + 1

    return Q
end

function explore(π, s)
    m = π.m
    n = π.n
    q = π.q
    c = π.c
    A = actions(m)

    bonus = (Nsa, Ns) -> Nsa == 0 ? Inf : sqrt(log(Ns)/Nsa)
    Ns = sum(n[(s,a)] for a in A)
    return argmax(a -> q[(s,a)] + c*bonus(n[(s,a)], Ns), A)

end

m = DenseGridWorld()

S = statetype(m)
A = actiontype(m)

# These would be appropriate containers for your Q, N, and t dictionaries:
n = Dict{Tuple{S, A}, Int}()
q = Dict{Tuple{S, A}, Float64}()
t = Dict{Tuple{S, A, S}, Int}()
c = 2

```

```
mutable struct MCTS
    m
    n
    q
    t
    c
end

π = MCTS(m,n,q,t,c)

# This is an example state - it is a StaticArrays.SVector{2, Int}
s = SA[19,19]

@assert s isa statetype(m)

for _ in 1:7
    simulate_MCTS(π, s)
end

inchrome(visualize_tree(π.q, π.n, π.t, s))
```

Problem 5:

5.1 Code For Problem 5

Using my MCTS code from problem 4 I developed two new functions, policy_, and eval_online_MCTS. The function policy_ selects an actions based on the Q estimate provided by 1000 iterations of MCTS with a depth of 20. The function eval_online_MCTS runs the 100 100-step MC trials of our functions, terminating when a terminal state is reached. This resulted in a mean discounted reward of -17.02 and $SEM = 7.77$

```
using DMUStudent.HW3: HW3, DenseGridWorld, visualize_tree
using POMDPs: actions, @gen, isterminal, discount, statetype, actiontype,
   simulate, states, initialstate
using D3Trees: inchrome, inbrowser
using StaticArrays: SA
using Statistics: mean, std
using BenchmarkTools: @btime

#############
# Question 5
#############


### rollout sim from state s0
function rollout(π, policy_function, s0, max_steps=20)
    r_total = 0.0
    t = 0
    s = s0
    while !isterminal(π.m, s) && t < max_steps
        a = policy_function(π, s) # replace this with a policy
        s, r = @gen(:sp,:r)(π.m, s, a)
        r_total += discount(π.m)^t*r
```

```
        t += 1
    end
    return r_total # replace this with the reward
end


### Policy for DenseGridWorld
function smart_policy(m,s)
    x,y = s

    AS = [
        (:down, (x, y-1)),
        (:up, (x, y+1)),
        (:right, (x+1, y)),
        (:left, (x-1, y))
    ]


    # Find closest goal from CURRENT state
    goals = [[20,20], [20,40], [40,20], [40,40]]
    closest_goal_cell = goals[argmin([(x - g[1])^2 + (y - g[2])^2 for g in
        goals])]
    #@show s
    #@show closest_goal_cell

    # Find which next state is closest to that goal
    distances_to_goal = [(next_s[1] - closest_goal_cell[1])^2 + (next_s[2]
        - closest_goal_cell[2])^2 for (a,next_s) in AS]

    #@show distances_to_goal
    #@show AS[argmin(distances_to_goal)][1]
    return AS[argmin(distances_to_goal)][1]

end

### main MCTS simulation
function simulate_MCTS(π, s, d = 100)

    # Unpack
    m = π.m

    if d <= 0 # Base case
        return rollout(π, smart_policy, s) # U(s)
    end

    A = actions(m)
    γ = discount(m)

    # If we haven't checked the actions at state s yet, assign all dict
       values to 0
    if !haskey(π.n, (s, first(A)))
        for a in A
            π.n[(s,a)] = 0
            π.q[(s,a)] = 0.0
```

```
73
74          end
75          # Value funciton estimate
76          return rollout(π, smart_policy, s)
77      end
78
79      # Explore decision tree
80      a = explore(π,s)
81      sp, r = @gen(:sp,:r)(m, s, a)
82      Q = r + γ * simulate_MCTS(π, sp, d-1)
83
84      π.n[(s,a)] += 1
85      π.q[(s,a)] += (Q-π.q[(s,a)])/π.n[(s,a)]
86      tval = get(π.t, (s, a, sp), 0)
87      π.t[(s,a,sp)] = tval + 1
88
89      return Q
90  end
91
92  ### Explore decision tree for mcts
93  function explore(π, s)
94      m = π.m
95      n = π.n
96      q = π.q
97      c = π.c
98      A = actions(m)
99
100     bonus = (Nsa, Ns) -> Nsa == 0 ? Inf : sqrt(log(Ns)/Nsa)
101     Ns = sum(n[(s,a)] for a in A)
102     return argmax(a -> q[(s,a)] + c*bonus(n[(s,a)], Ns), A)
103
104 end
105
106 ### Get an action based on MCTS tree
107 function policy_(π, s)
108     for k in  1:1000
109         simulate_MCTS(π,s) # 1000 iterations to choose each action, d =
                100 by default
110     end
111     return argmax(a -> π.q[(s,a)], actions(π.m))
112 end
113
114 function eval_online_MCTS(m)
115     all_rewards = []
116
117     for i in 1:100 # 100 MC simulations
118
119         # Declare/Redeclare necessary params
120         n = Dict{Tuple{S, A}, Int}()
121         q = Dict{Tuple{S, A}, Float64}()
122         t = Dict{Tuple{S, A, S}, Int}()
123         c = sqrt(2)
124
125         π = MCTS(m ,n ,q ,t , c)
```

```
126
127         s = rand(initialstate(m))
128         total_reward = 0.0
129
130         for j in 1:100 # 100 step unless it reaches a terminal state
131             a = policy_(π,s) # Calls MCTS to plan next actions
132
133             sp, r = @gen(:sp,:r)(π.m, s, a) # Take the next step
134
135             total_reward += r
136
137             s = sp
138
139             if isterminal(m,s) break end# Break out, run next trial
140
141         end
142
143         push!(all_rewards, total_reward)
144
145     end
146
147     return all_rewards
148
149 end
150
151 m = DenseGridWorld()
152
153 S = statetype(m)
154 A = actiontype(m)
155
156 # These would be appropriate containers for your Q, N, and t dictionaries:
157 n = Dict{Tuple{S, A}, Int}()
158 q = Dict{Tuple{S, A}, Float64}()
159 t = Dict{Tuple{S, A, S}, Int}()
160 c = sqrt(2)
161
162 mutable struct MCTS
163     m
164     n
165     q
166     t
167     c
168 end
169
170 results = eval_online_MCTS(m)
171
172 @show mean(results)
173 @show std(results)/sqrt(100)
```

Problem 6:

For the challange problem, I mainly focused on tuning the value of c so that my code would reach a score of 50. I opted to stick with the MCTS code developed in problem 4, as well as the rollout

policy I implemented in problem 3. I modified the rollout policy to account for the added goal cells that came with the larger grid world. While this rollout policy may be slow, as it includes loops over the goal cells and actions, I wanted a policy that was likely close to the optimal policy, and it likely did not cause much of a slow down in compute. As mentioned earlier, I mainly focused on tuning c for my MCTS code to meet the requirements. I began with the value I initially started with, $c = \sqrt{2}$ and increased the order by 10 until it achieved a score of 50. I chose to increase the value of c instead of decreasing it because I believed that encouraging the tree to explore other actions would result in the tree search trying the worse actions more often, get a lower value for Q, and ultimately end up choosing the best action in the function policy_. I kept iterating this process until I recieved a score of 50, which I achieved with a value of $c = 100\sqrt{2}$. While the following value for c was able to get me a score of 50, once I turned the time flag on, my code ran too slow. Thus I continued iterating the value, this time progressivly decreasing it by a value of 10, starting at 100 until I got desirable results. I did this because I found it to have a better performance on the runtime when using the @btime command. Finally, I lowered my depth to a value of 4, and ended with a score of 91.6.

6.1 Code For Problem 6

```
using DMUStudent.HW3: HW3, DenseGridWorld, visualize_tree
using POMDPs: actions, @gen, isterminal, discount, statetype, actiontype,
    simulate, states, initialstate
using D3Trees: inchrome, inbrowser
using StaticArrays: SA
using Statistics: mean, std
using BenchmarkTools: @btime
#############
# Question 6
#############

const goals = ((20,20), (20,40), (20,60), (20,80), (40,20), (40,40),
    (40,60), (40,80), (60,20), (60,40), (60,60), (60,80), (80,20), (80,40),
     (80,60), (80,80))

### rollout sim from state s0
function rollout(π, policy_function, s0, max_steps=100)
    r_total = 0.0
    t = 0
    s = s0
    while !isterminal(π.m, s) && t < max_steps
        a = policy_function(π, s) # replace this with a policy
        s, r = @gen(:sp,:r)(π.m, s, a)
        r_total += discount(π.m)^t*r
        t += 1
    end
    return r_total # replace this with the reward
end


### Policy for DenseGridWorld
function smart_policy(m,s)
    x,y = s

```

```
    AS = [
        (:down, (x, y-1)),
        (:up, (x, y+1)),
        (:right, (x+1, y)),
        (:left, (x-1, y))
    ]

    # Find closest goal from CURRENT state
    closest_goal_cell = goals[argmin([(x - g[1])^2 + (y - g[2])^2 for g in goals])]
    #@show s
    #@show closest_goal_cell

    # Find which next state is closest to that goal
    distances_to_goal = [(next_s[1] - closest_goal_cell[1])^2 + (next_s[2] - closest_goal_cell[2])^2 for (a,next_s) in AS]

    #@show distances_to_goal
    #@show AS[argmin(distances_to_goal)][1]
    return AS[argmin(distances_to_goal)][1]

end

### main MCTS simulation
function simulate_MCTS(π, s, d = 4)

    # Unpack
    m = π.m

    if d <= 0 # Base case
        return rollout(π, smart_policy, s) # U(s)
    end

    A = actions(m)
    γ = discount(m)

    # If we haven't checked the actions at state s yet, assign all dict values to 0
    if !haskey(π.n, (s, first(A)))
        for a in A
            π.n[(s,a)] = 0
            π.q[(s,a)] = 0.0

        end
        # Value funciton estimate
        return rollout(π, smart_policy, s)
    end

    # Explore decision tree
    a = explore(π,s)
    sp, r = @gen(:sp,:r)(m, s, a)
    Q = r + γ * simulate_MCTS(π, sp, d-1)

    π.n[(s,a)] += 1
```

```
    π.q[(s,a)] += (Q-π.q[(s,a)])/π.n[(s,a)]
    tval = get(π.t, (s, a, sp), 0)
    π.t[(s,a,sp)] = tval + 1

    return Q
end

### Explore decision tree for mcts
function explore(π, s)
    m = π.m
    n = π.n
    q = π.q
    c = π.c
    A = actions(m)

    bonus = (Nsa, Ns) -> Nsa == 0 ? Inf : sqrt(log(Ns)/Nsa)
    Ns = sum(n[(s,a)] for a in A)
    return argmax(a -> q[(s,a)] + c*bonus(n[(s,a)], Ns), A)

end

### Get an action based on MCTS tree
function select_action(m, s)

    S = statetype(m)
    A = actiontype(m)

    # These would be appropriate containers for your Q, N, and t
       dictionaries:
    n = Dict{Tuple{S, A}, Int}()
    q = Dict{Tuple{S, A}, Float64}()
    t = Dict{Tuple{S, A, S}, Int}()
    c = 70

    π = MCTS(m,n,q,t,c)

    Threads.@threads for _ in  1:1000
        simulate_MCTS(π,s) # 1000 iterations to choose each action, d =
           100 by default
    end
    return argmax(a -> π.q[(s,a)], actions(π.m))
end

m = DenseGridWorld()

mutable struct MCTS
    m
    n
    q
    t
    c
end


```

```
S = statetype(m)
A = actiontype(m)

n = Dict{Tuple{S, A}, Int}()
q = Dict{Tuple{S, A}, Float64}()
t = Dict{Tuple{S, A, S}, Int}()
c = 70

π = MCTS(m,n,q,t,c)

s = rand(initialstate(m))

#@btime simulate_MCTS(π,s)

HW3.evaluate(select_action,"bijan.jourabchi@colorado.edu",time=true)
```